

배터리 제약 MEC에서: 어느 사용자를 오프로딩할까

Energy-Aware Multi-Server Mobile Edge Computing: A Deep Reinforcement Learning Approach

Navid Naderializadeh, Morteza Hashemi (2019)

여러 MEC 서버가 각자의 사용자 중 한 명을 고르는 multi-agent DQN

왜 ‘모두 오프로딩’이 답이 아닌가

공유 무선 자원과 서버 queue 때문에 집중 오프로딩은 처리량을 낮출 수 있다

- | | | |
|----|--------|-------------------------------|
| 01 | 여러 사용자 | battery와 FIFO task queue를 가진다 |
| ↓ | | |
| 02 | 공유 무선 | 동시에 보내면 통신 자원을 나눈다 |
| ↓ | | |
| 03 | 서버 부담 | 집중된 task는 queue와 연산 부담을 만든다 |
| ↓ | | |
| 04 | 동적 선택 | 이번 slot에 보낼 사용자를 고른다 |

문제는 서버 선택 자체보다, 각 서버가 제한된 자원을 누구에게 쓸지 정하는 것이다

이 논문이 실제로 결정하는 범위

일반적인 전역 resource allocation이 아니라, 미리 연결된 pool 안의 한 사용자 선택이다

항목	논문의 설정	slot 변화	범위
association	가장 큰 long-term gain의 server	아니오	매 slot server migration이 아님
user pool	server마다 서로 겹치지 않는 U_i	아니오	서버들이 같은 사용자를 다투지 않음
agent action	자기 pool에서 한 사용자 선택	예	사용자마다 server를 고르는 action이 아님
나머지 사용자	energy가 되면 local computing	예	모두 offload하는 구조가 아님

각 MEC agent는 자신의 disjoint pool에서 한 명을 고른다

한 time slot에서 변하는 상태와 행동

task는 FIFO queue에 쌓이고, battery와 channel은 다음 선택의 입력이 된다

상태	뜻	이번 slot의 영향	다음 slot
queue	도착한 homogeneous task의 FIFO buffer	local 또는 offload한 task가 빠짐	새 arrival와 남은 task가 관측됨
energy E_j	slot 시작 battery energy	idle, local, radio energy가 빠짐	작아지면 feasible power가 줄어듦
uplink SNR	현재 uplink 품질	offload bit capacity를 바꿈	새 관측값으로 들어감
waiting time	queue 안 task들의 평균 대기	선택 priority의 관측 feature	service와 arrival에 따라 바뀜

정책은 고정 rule이 아니라, queue·energy·waiting time·SNR의 현재 관측에 반응한다

local computing: battery가 CPU capacity도 제한한다

hardware 최대 주파수와 slot 에너지 예산 중 더 엄격한 쪽이 local service를 정한다

- | | | |
|----|-------------------|---------------------------------|
| 01 | slot 시작 energy | $E_j(t)$ 와 slot 길이 τ 를 본다 |
| ↓ | | |
| 02 | feasible power | $E_j(t) / \tau$ 를 넘지 않는다 |
| ↓ | | |
| 03 | CPU frequency cap | hardware cap과 energy cap 중 작은 값 |
| ↓ | | |
| 04 | FIFO service | capacity 안에 드는 head task부터 처리 |

battery가 작아지면 local CPU frequency와 처리 가능한 bit 수도 함께 줄어든다

offloading: battery-SNR가 uplink bit cap을 정한다

선택되었다고 해서 queue 전체를 보내는 것이 아니라, 이번 slot rate가 허용한 만큼만 보낸다

01 selected user server agent가 pool에서 한 명을 고른다



02 feasible TX power battery budget과 hardware cap을 함께 적용



03 uplink rate bandwidth와 SNR로 rate를 계산



04 FIFO offload rate가 허용한 bit만 head부터 전송

offload capacity는 선택 여부만이 아니라, slot의 channel과 남은 energy에도 달려 있다

energy transition과 두 평가 지표

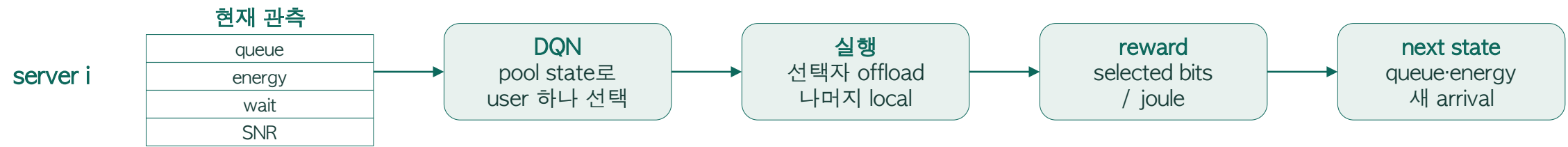
빠른 service는 battery를 더 쓸 수 있으므로, completion time과 lifetime은 충돌할 수 있다

항목	한 slot 후 변화	정의	주의
idle	$E(next)=E-\epsilon$	standby energy도 소비	아무것도 안 해도 battery는 감소
local / offload	$E(next)=E-\text{action-}\epsilon$	각 action의 에너지를 뺌	local과 radio energy는 다름
system lifetime	첫 battery depletion에서 종료	모든 user가 살아 있는 마지막 slot	평균 battery lifetime이 아님
task completion time	완료 slot - arrival slot	종료 전 완료 task의 평균	기본 단위는 slot 수

빠른 task service는 battery를 더 써 lifetime과 충돌할 수 있다

한 MEC server가 보는 DQN feedback loop

아래는 한 server의 serial view이며, disjoint pool의 다른 agents는 같은 slot에 병렬로 선택한다



reward는 selected user's bit/J이고, lifetime completion time은 별도다

학습 reward와 논문이 보고한 KPI를 분리해 읽기

두 종류의 수치를 같은 목적함수로 합쳐 썼다고 가정하면 논문의 설명 범위를 넘는다

구분	무엇을 재나	범위	논문에서의 역할
immediate reward	선택 user의 bit / offload energy	한 agent, 한 slot	DQN action learning signal
system lifetime	첫 battery depletion 전까지의 slot	모든 users	trade-off 평가
completion time	arrival부터 completion까지	완료된 tasks	trade-off 평가
해석	reward와 KPI는 관련될 수 있음	실험 설정에 의존	동일한 scalar objective는 아님

DQN이 bit/J를 학습한다고 해서 lifetime과 completion time의 최적성이 증명된 것은 아니다

교육용 한 slot 계산: 선택 뒤 무엇이 다음 state가 되나

가정한 숫자이며 논문의 simulation 측정값이나 실제 DQN action이 아니다

가정	이번 slot action	계산	next observation
user A: queue 2, $E=4$ J	local로 1 task 처리	$E(\text{next})=4-0.1-\text{epsilon}$	queue: $2-1+1=2$
user B: queue 3, $E=2$ J	가상의 policy가 B를 offload 선택	$2,000 \text{ bit} / 0.2 \text{ J} = 10,000 \text{ bit/J}$	$E(\text{next})=2-0.2-\text{epsilon}$
toy task size 1,000 bit	B가 2 task를 전송	queue: $3-2+0=1$	B queue와 energy가 새 input

선택은 한 번의 reward만 만들지 않고, queue·energy·waiting time을 바꿔 다음 선택을 바꾼다

episode와 DQN training은 어떻게 진행되는가

network를 무작정 반복하는 것이 아니라, first battery depletion까지의 episode에서 experience를 모은다

01 episode reset user·server 위치와 buffer·energy를 초기화



02 slot action 각 server agent가 user 하나를 선택



03 state transition service, energy, arrival로 state를 갱신



04 episode end 첫 battery depletion 뒤 다음 layout으로 reset



05 DQN update replay buffer의 experience로 network를 갱신

종료 조건은 모든 user의 평균 battery가 아니라 한 user의 battery depletion이다

보고된 simulation 조건: 결과를 떼어 읽지 않기

아래 수치는 deployment specification이 아니라 이 논문의 numerical setting이다

범주	설정	범주	설정
area / slot	10 m × 10 m / 100 ms	wireless	20 MHz, FDMA, noise -174 dBm/Hz
user energy	initial max energy: uniform (0.01, 1) J	task	mean arrival 10, size 1 KB
hardware	user CPU 1 GHz; server CPU 3 GHz	power	max TX 27 dBm; kappa 10^{-27}
DQN	2 tanh layers, each 200 nodes	exploration	100 episode random, then 1%까지 decay

reported trade-off는 이 FDMA simulator와 training setting에 조건부다

비교 baseline은 어떤 사용자만 우선하는가

두 greedy rule은 같은 two-KPI trade-off의 양끝을 보여 주는 비교 기준이다

방법	server가 고르는 user	직관	예상되는 편향
Time-Greedy	average queue waiting time이 가장 큰 user	오래 기다린 task를 먼저 처리	completion time을 낮추려 battery를 더 빨리 쓸 수 있음
Energy-Greedy	energy level이 가장 낮은 user	battery가 약한 user를 우선 보호	service가 늦어져 completion time이 커질 수 있음
DQN	현재 observation으로 learned selection	queue·energy·wait·SNR을 함께 관측	두 지표를 동시에 지배한다고 가정할 수 없음

baseline은 서로 다른 priority의 extreme을 보여 주는 비교 기준이다

Figure 4: training 중 server 수가 바뀌는 모습

5 users에서 1·2·3 servers를 비교한 learning curves이며, 정확한 tabulated 수치가 아니다

조건	보이는 비교	저자 해석	읽을 때 주의
5 users, 1/2/3 servers	system lifetime과 completion time의 DQN curves	약 1,000 episodes 뒤 convergence	plot에서 임의의 정확한 수치를 만들지 않음
더 많은 servers	curves의 상대적 변화	load balancing과 overall performance 개선	다른 area·arrival에도 보장된다는 뜻은 아님
training episode	learning progress의 x-axis	policy가 안정되는 과정	물리적 seconds나 slot time axis가 아님

Figure 4는 이 setup의 learning curve이지 general scaling law가 아니다

Figure 5의 핵심: 한 방법이 두 축을 모두 이기지 않는다

source plot의 3·4·5 server 비교에서 DQN은 two greedy extremes 사이의 trade-off를 보인다

방법	system lifetime	task completion time	해석
Time-Greedy	그림에서 더 짧은 편	그림에서 더 낮은 편	대기시간 priority의 fast-service 쪽 extreme
Energy-Greedy	그림에서 더 긴 편	그림에서 더 높은 편	battery priority의 lifetime 쪽 extreme
DQN	두 extreme 사이	두 extreme 사이	저자가 말한 balanced trade-off
조건 주의	Figure 5 x-axis: 3, 4, 5 servers	본문 p.4는 3 servers fixed 라고 서술	원문 내부 조건 불일치

DQN은 이 simulation에서 두 extreme 사이의 trade-off를 보인다

이 논문에서 얻을 수 있는 것과 아직 남는 것

mechanism은 분명하지만, result의 범위는 작은 simulator와 stated action space에 묶여 있다

- fixed long-term association 뒤의 per-server one-user selection을 설명한다
- battery·queue·SNR가 next-slot 선택으로 되먹임된다
- $10\text{ m} \times 10\text{ m}$ simulator 밖 deployment 성능은 직접 측정하지 않는다
- seed·confidence interval·error bar가 없어 significance를 읽을 수 없다
- selected-user bit/J reward는 two-KPI 최적성 theorem이 아니다

제한된 action space에서 energy-aware selection의 trade-off를 보였다

논문의 논리 사슬을 한 장으로 정리

shared wireless MEC의 병목을 per-server learned selection과 state feedback으로 다룬다

01	문제	모두 offload하면 wireless와 server burden이 커진다
↓		
02	제약	FIFO, CPU/radio cap, battery transition
↓		
03	정책	server별 DQN이 pool 안 한 user를 선택
↓		
04	feedback	queue·energy·arrival가 next observation을 만든다
↓		
05	결과	greedy extremes 사이의 conditioned trade-off

정책은 server 하나의 현재 pool과 next-slot state를 단위로 한다